

Three open questions, explained in full, with the experiment status and a TODO

For Halil, 2026-08-27

Why this document exists

You asked for three things. You asked what the first item actually says, because the one sentence I wrote did not explain it. You asked where our proof needs an additional argument, which is the second item. You asked me to correct the timing and give the number, which is the third item and which I have now done. You then asked for the experiment status and a list of what is left.

I have also taken your point about space. Material moves to the technical report because the submission has ten pages and the material does not fit. That is the right reason and nothing below argues against it. Where I raise a problem with a cut, the problem is never that the material moved. It is that in a small number of places the submission keeps a claim while the sentence that supports it moved, and does not tell the reader where it went. The fix is a pointer, not a restoration.

Item 1. Two numbers in the introduction do not say what we say they say

What the paper currently says

The introduction, at `docs/paper/sections/intro.tex` lines 19 to 22, reads as follows.

An observer of the per-round updates mounts membership inference at 87% accuracy on one model and dataset, where the same attack against the final model alone falls to 54.5%.

This sentence is the empirical anchor of the paper’s motivation. It is the evidence for the claim, made one sentence earlier, that federated learning’s privacy risk is concentrated at training time. If it does not hold, the argument for a one-shot protocol loses its quantitative support.

What the cited paper actually reports

Both numbers are in Nasr, Shokri and Houmansadr, IEEE S&P 2019. Neither describes what our sentence describes.

The figure of 87% is 87.3%, and it comes from their Table X. It is the accuracy of an adversary that their paper classifies as **global, active and isolating**. The word isolating names a specific deviation. The parameter server withholds the other clients’ updates from the target client, so that the target trains against a model shaped only by its own data and overfits it more sharply. The server then measures the result. That is a server that departs from the protocol in order to attack.

Our threat model does not include such a server. `docs/paper/sections/security.tex` states that the server is semi-honest, which means it follows the protocol and only tries to infer from what it legitimately sees. So the introduction motivates our design by citing an adversary strictly stronger than the one the paper defends against. A reviewer who knows the Nasr paper will notice this, and the objection is not easy to answer, because we chose the semi-honest model deliberately.

The number for an adversary that only observes, which is what the word observer means, is in the same table. The **global passive** attacker reaches 79.2% on the same model and the same dataset. That is the number our sentence should be quoting if the sentence keeps the word observer.

The figure of 54.5% has a different problem. It is not the same attack. It appears once in the Nasr paper, in their introduction, and it describes a **black-box** attack against a **stand-alone** pre-trained model, which is a model that was never trained federatively at all. Our sentence calls it “the same attack against the final model alone”, and it is neither the same attack nor a federated setting.

There is a further difficulty with 54.5% that is worth knowing before you choose. Their own Table VIII reports 67.7% for the black-box attack on that same stand-alone CIFAR-100 DenseNet. The paper’s introduction and the paper’s table disagree with each other. We currently quote the smaller of the two, which happens to be the one that makes our contrast look larger.

Why this matters in practice

The sentence as written compares an active federated attack against a passive centralised one and calls them the same attack. A reviewer who opens the cited paper finds three separate discrepancies. Two of them make our motivating gap look wider than the source supports.

Your options, with what each costs

The first option is to use the passive federated number and stop claiming the two attacks are the same. The sentence would then say that a parameter server which observes the per-round updates reaches 79%, where a black-box attack against the fully trained model reaches 54.5%. This keeps a quantitative contrast, it matches our own semi-honest server, and every number then describes what the source says it describes. The contrast narrows from thirty-three points to twenty-five.

The second option is to keep 87.3% and add the qualifier. The sentence would have to say that the server reaches that accuracy by isolating a target client during training. This is honest, and it costs us the awkwardness of citing a malicious server in the motivation for a semi-honest design.

The third option is the most conservative. Use 79.2% against Table VIII’s 67.7%, so that both numbers come from tables rather than from the source’s introduction. The contrast then narrows to twelve points, which is a real gap but a much weaker motivating sentence.

My recommendation is the first option. It is the best balance between a contrast strong enough to motivate the paper and a claim that survives a reviewer opening the source. If you want the sentence to be unimpeachable rather than persuasive, take the third.

This is submission text and both spans need substitution, so `check_subseq.py` will flag it and it is your decision rather than mine. I have changed nothing.

Item 2. Where the security proof needs an additional argument

This is the item you said you did not follow, so I will build it from the beginning.

The property of CKKS that creates the problem

CKKS is an approximate scheme. Decrypting a CKKS ciphertext does not return the message m . It returns $m + e$, where e is the accumulated approximation error. The size and the shape of e depend on the circuit that was evaluated and on the plaintexts that went through it.

The standard IND-CPA game never gives the adversary a decryption. It gives it ciphertexts and asks it to distinguish. For an exact scheme such as BGV or BFV that is enough, because a decryption returns exactly m and reveals nothing beyond the value the protocol meant to output.

For CKKS it is not enough. Li and Micciancio showed at Eurocrypt 2021 that an adversary which sees both a ciphertext and the decryption of a computation on it can subtract the two, recover the error term e , and use a collection of such error terms to solve for the secret key. The attack is passive. It needs no deviation from the protocol. It needs only decryption outputs of honestly evaluated circuits, which is exactly what a protocol that answers queries hands out. The repaired security notion is called IND-CPA with a decryption oracle, written IND-CPA^D , and the standard countermeasure is to add extra noise to a value before releasing it, which the literature calls noise flooding or smudging. The flooding noise must be large enough that it statistically hides the circuit noise e , and how large that is depends on how much noise the circuit accumulated.

Where our protocol releases decryptions

Our protocol releases decryptions in two places, and this is what puts us in the IND-CPA^D setting rather than the IND-CPA setting.

The selection step decrypts one value, which is the index of the chosen arrangement. Every party learns it.

Every query ends with the quorum key-switching the encrypted label to the querying client, which then decrypts it. The querying client may be corrupt. In **Definition 4**, the content game, the adversary is explicitly allowed to ask queries through its own clients and to read the answers.

So we hand decryption outputs to parties that may be adversarial, repeatedly, on circuits whose noise depends on the honest clients' data.

The exact step in the proof that is not covered

Theorem 1 assumes the scheme is IND-CPA against an adversary holding $t - 1$ key shares, and its proof builds a simulator. The simulator replaces each honest client's uploaded ciphertext with an encryption of zero and argues that the adversary cannot tell, by a hybrid argument over IND-CPA. That part of the proof is sound and IND-CPA is the right assumption for it, because those ciphertexts are never decrypted for the adversary.

The step that IND-CPA does not cover is the one in the paragraph headed *Selection and serving*. There the simulator must also produce the decryption transcripts, which are the partial decryption shares and the decrypted values the adversary actually sees.

Consider what the adversary sees in the real protocol when it asks a query. It receives the decryption of the served label, which is $\hat{y} + e$, where e is the noise the serving circuit accumulated. That circuit applied the shared head θ^* , and θ^* is a function of the honest clients' uploaded displacements. So the noise term e carries a dependence on the honest clients' data.

Now consider the simulator. It is given the leakage and the answers that the functionality returns to the corrupted clients. It knows $\hat{y}$. It does not know the honest clients' data, so it cannot compute the e that the real execution would have produced. It must sample something instead.

For the simulation to be indistinguishable, what the adversary sees must not depend on the honest inputs in a way the adversary can detect. Smudging is precisely what buys that. If the released value is $\hat{y} + e + E$ where the flooding noise E is drawn much larger than e , then the distribution of $\hat{y} + e + E$ is statistically close to that of $\hat{y} + E$, which the simulator can sample without knowing anything about the honest clients. The proof step reads: the real and simulated decryption shares are statistically indistinguishable because the smudging noise dominates the circuit noise.

Without that step, and without an assumption that licenses it, the proof has a gap at exactly the point where the protocol delivers its output. That is the additional argument you asked about.

What our implementation already does, which is better than I first reported

I checked the code before writing this, and the situation is considerably better than the review I relayed to you last night suggested. That review said we smudge nowhere. That is wrong and I am correcting it.

We do smudge. `fhe/main.go` at line 337 and `fhe/serve.go` at line 193 both set a smudging parameter on the collective key-switch protocol, at eight times the fresh-encryption noise, with the bound set to six times that. The comment in `fhe/main.go` states the reasoning, which is that this is the choice the library's own multiparty tests use.

So the protocol is not broken and the implementation is not missing the countermeasure. What is missing is threefold and all of it is documentation and analysis rather than code.

Neither document mentions smudging, noise flooding, or IND-CPA^D anywhere. I grepped both. A cryptography reviewer will look for exactly these words and not find them.

The parameter is the library's default rather than a value derived for our circuit. Eight times the fresh-encryption noise is a sensible engineering default. It is not the output of an analysis that asks how much noise our argmax accumulates over nine to thirty-four refreshes and what flooding magnitude statistically hides it at our security parameter. Whether eight is enough for our circuit is, at this moment, **open**. It is very likely too small if one wants a formal statistical-distance bound, because the required flooding typically scales with the circuit noise multiplied by a factor exponential in the statistical security parameter.

Theorem 1 states IND-CPA where it should state IND-CPA^D, and its proof has no step covering the released decryptions.

The one piece of good news, which is worth stating in the paper

There is a property of our design that makes this much less painful than it would be for a system that returns real-valued scores, and I think it is a strength we should claim rather than a problem we should hide.

We return an argmax index, not a logit vector. An index is an integer. Flooding noise that is small relative to one half does not change the value after rounding. A system that returned scores would have to trade flooding magnitude against output precision directly, because the noise lands on the number the user reads. We do not. Our output is discrete, so we can flood far more aggressively than a score-returning system before the answer changes at all.

This also tells us what the measured claim that the encrypted argmax is exact, with a maximum absolute error of 0.0000 in `results/fhe_serve/argmax_tournament.csv`, is worth. It was measured at the library’s default smudging. Whether it survives smudging sized for a formal IND-CPA^D bound is **not yet measured**, and it is a cheap thing to measure, because it means re-running the tournament with a larger sigma and checking the index still comes out right.

What I suggest, and what it costs

The change to **Theorem 1** is two clauses. Assume the scheme is IND-CPA^D and state that partial decryptions are smudged. The change to the proof is one sentence in the serving paragraph, saying the simulated decryption share is statistically indistinguishable because the flooding noise dominates the circuit noise. Two citations enter the bibliography, which are Li and Micciancio for the attack and the notion, and Mouchet et al. for the multiparty protocols that already carry the smudging step, and that key is already in our bibliography.

That is an addition rather than a rewrite, so it does not fight the subsequence rule. It touches a security claim, so it is yours.

The experiment that would close it is to re-run the argmax at a smudging sigma derived from the circuit noise and confirm the index is still exact. That is Go work of an hour and compute of minutes.

Item 3. The timing, corrected, with the numbers

You asked me to correct this and to give the number. I have done it and the report is rebuilt.

What was wrong

The paper reports that one query takes 31.5 to 113.2 seconds. Those figures come from `results/fhe_serve/argmax_tournament.csv`. That file records a tournament whose sign evaluation is, in the words of the case README, collective-refresh-backed. It spends nine collective refreshes at four classes and thirty-four at a hundred classes.

The paper also reports that one query costs 13.5 MiB of traffic. That figure prices a different design. The paragraph in `sec:exp-cost` says so plainly, that restoring levels by collective refresh would cost roughly 1.6 GiB of traffic for a single label at a hundred classes, that generating bootstrapping keys once and letting the server restore levels alone reduces that to zero, and that the protocol therefore specifies the second.

So the latency describes the collective-refresh design and the traffic describes the bootstrapping-key design, and the design the protocol specifies has never been timed. The two figures sit one paragraph apart in the same subsection and in the abstract they sit in the same sentence.

The numbers

From `results/fhe_serve/argmax_tournament.csv`, at ten parties and ring degree 2^{15} .

classes	refreshes	total	in refresh	local evaluation
4	9	31.2 s	12.3 s	19.0 s
6	14	47.3 s	18.9 s	28.4 s
14	19	63.7 s	25.9 s	37.9 s
77	34	112.4 s	46.3 s	66.1 s
100	34	113.0 s	46.3 s	66.6 s

The paper’s 31.5 and 113.2 are these totals plus the 0.24 seconds of arithmetic and key switching that the paper accounts separately, so they reconcile exactly. The collective refresh is between thirty-nine and forty-one per cent of the total at every label-space size.

The bootstrapping keys in `results/fhe_serve/btp_keys.json` are generated at ring degree 2^{16} , not 2^{15} , so the two designs also differ in ring degree.

What I changed

I added one report-only paragraph to `sec:exp-cost`, immediately after the paragraph that states the design decision. It says that the reported figures were measured with collective-refresh-backed level restoration and not with the bootstrapping keys the protocol specifies, gives the refresh counts and the split above, says that the local-evaluation component is common to both mechanisms, says that the specified design replaces the refresh component with a server-side bootstrap that we have not timed, notes the ring-degree difference, and states that we do not claim these figures for the specified variant.

All nine gates pass. The submission is unchanged.

What is still open here, and it is your call

The submission’s abstract says the protocol answers one query in 31.5 to 113.2 seconds for 13.5 MiB of traffic. After the correction above, the report says those two numbers describe different level-restoration mechanisms. The abstract does not. Fixing that in the submission means either adding a qualifying clause, which is submission text and needs your approval, or running the measurement so that both numbers describe the specified design.

I recommend running the measurement. It is the only one of these three items that a reviewer can check by reading our own appendix against our own abstract, and the work is bounded.

Experiment status

Everything below is the state of the repository as of 2026-08-27. A row marked done has a file under `results/` that holds the number.

Done and in the paper

what	record
accuracy on five tasks, four quantities each the pooled reference CIFAR-10 on the DENSE and FedAUXfdp partitions, whole test set accuracy against client count, skew, local steps	results/personal_adapter*/stratified/results.csv results/centralised_ceiling/results.csv results/personal_adapter_vision/cifar10_matched_full
encrypted argmax cost, tournament and sequential fold per-query and setup communication bootstrapping key size and generation time protocol operation costs over ring degree and party count	results/personal_adapter/nsweep.csv, sensitivity.csv results/fhe_serve/argmax_tournament.csv, argmax_cost.csv results/fhe_serve/comm_grid.json results/fhe_serve/btp_keys.json results/fhe_serve/cost_grid.json

Done and report-only

what	record
extraction cost against query budget the extraction scaling law in C and d the noise-on-labels defence, newly written up the selection rule against the held-out vote	results/extraction_budget/results.csv results/extraction_scale/results.csv results/extraction_defence/results.csv results/personal_adapter/nsweep.csv, sensitivity.csv

Not done

what	why it matters	size
membership inference on the served interface	the paper’s privacy story currently argues extraction cost only and never says what a copy reveals about a record	a VALAR session, after the mia/ rewire
latency under the specified bootstrapping variant the argmax at IND-CPA ^D -sized smudging	the abstract’s two numbers describe different designs tells us whether “the encrypted argmax is exact” survives a formal flooding parameter	Go work, minutes of compute an hour of Go, minutes of compute
the cost of the selection phase	there is no number anywhere, and the submission asserts it is a bounded exchange	either a stated probe cap, which makes it arithmetic, or a measurement

what	why it matters	size
one real end-to-end encrypted query	<code>fhe/main.go</code> has no flag that loads a real head, so every serving benchmark runs on synthetic vectors while the introduction says the protocol is implemented rather than simulated	a Python exporter and about a hundred lines of Go
the twelve CIFAR-10 cells folded into the selection table	turns the selection result from 13 of 15 into 24 of 27	free, no compute

Stale, and now labelled as such

The membership suite in `mia/` and the cases `results/heifd_021_mia` and `results/heifd_mia_freeze_a` attack a released plaintext model and a Phase-0 prototype channel. The current method has neither. I put a dated header on `mia/README.md` and a durable note at `results/heifd_021_mia/PRE-PIVOT.md`, because the `README.md` in that directory is regenerated by `mia.report` and a note written into it would be overwritten. Those numbers remain usable as the disclosure counterfactual and as nothing else.

TODO

Ordered by what blocks what, not by size.

Decisions only you can make

1. Choose an option for the Nasr sentence in the introduction. Item 1 above lists three, with what each costs.
2. Decide whether **Theorem 1** moves to IND-CPA^D with a smudging clause, and whether the proof gains the one step. Item 2 above gives the exact wording of the gap.
3. Decide whether the abstract’s latency figure gets a qualifying clause or waits for the measurement.
4. Decide the selection-phase question. Either the method states a probe cap per client, which makes the cost arithmetic on numbers we already hold, or the method says the per-class accuracies are computed locally on plaintext logits with only the counts encrypted. The second is a change to what the method claims.

Compute, all AFK on VALAR

5. Rewire `mia/target.py` to the current pipeline and run the membership measurement as a white-box attack on the true merged head, swept over the query budget. **Proposition 2** makes this a ceiling on every budget, which is why the extract-then-attack pipeline no longer has to be built.
6. Re-run the argmax tournament under server-side bootstrapping keys, so that the latency and the traffic describe the same design.
7. Re-run the argmax at a smudging sigma derived from the circuit noise and check the index is still exact.

8. Export one real trained head and run one query end to end.

Free, no compute

9. Fold the twelve CIFAR-10 cells into the selection table.
10. Add the pointers where the submission keeps a claim whose support moved to the report. There are four that I found. **Theorem 2** says Section V-E turns δ into a bound on Q and the paragraph that does it is report-only. The setup says the paper reports both differences and the paragraph reporting the second is report-only. The extraction result reports a fidelity curve and the definition of fidelity is report-only. The novelty claim’s one counterexample is stated only in the report. Each is one `\trsee` and costs a few words, which is the cheapest thing on this list and the one a reviewer is most likely to trip over.